Naive substring search and KMP

Finding the first match in text

Ethan Goldman

Advanced Algorithms 41052

What are we searching for?

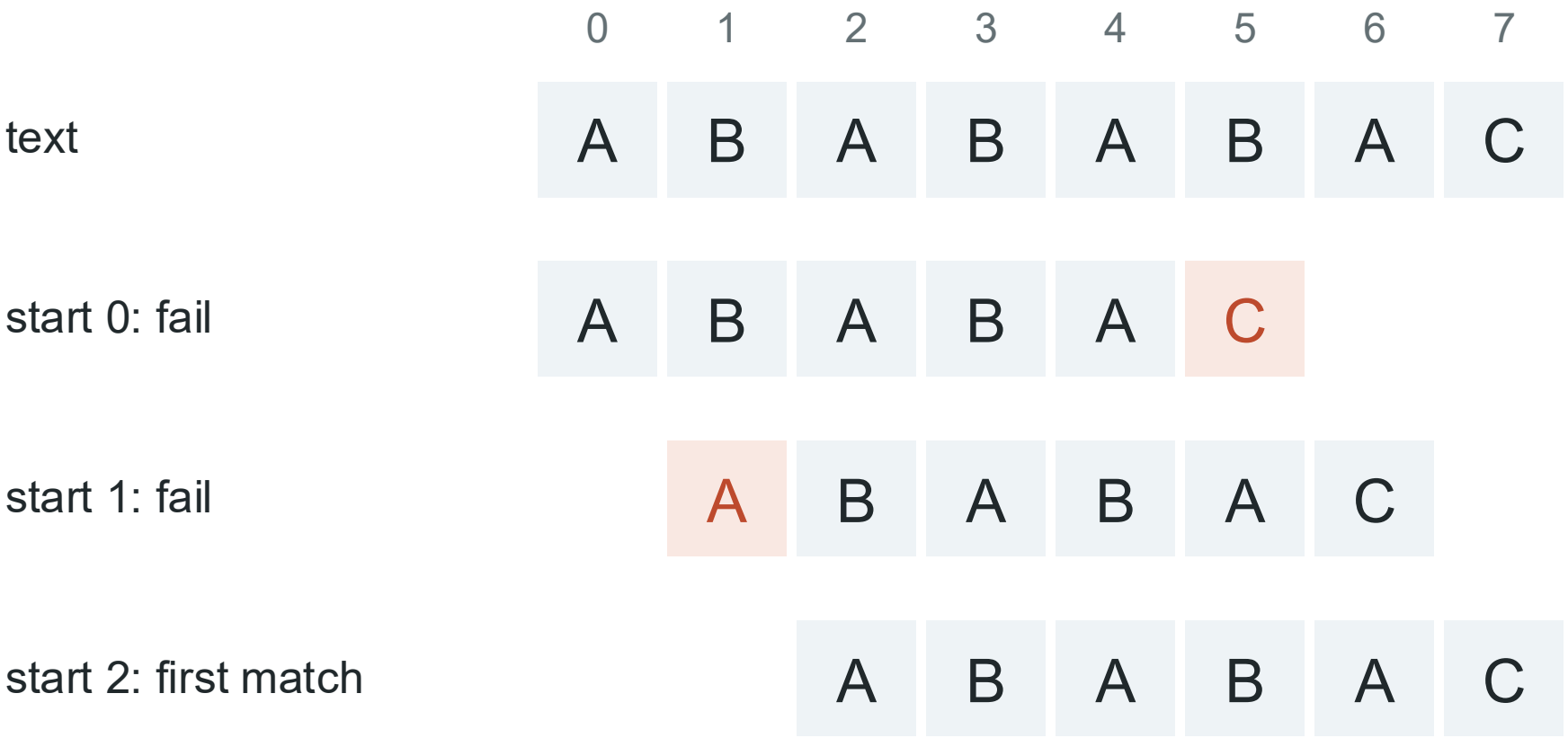
Find where the pattern first appears

No match: return npos Empty pattern: return 0

The program works with bytes, including zero bytes

ABABABAC contains ABABAC at position 2

Naive search: try each starting point



13 comparisons here. Long near-matches repeat work.

KMP: finding repetition in the pattern

LPS records which beginning also appears at the end

index	0	1	2	3	4	5
pattern	A	B	A	B	A	C
LPS	0	0	1	2	3	0

ABABA starts and ends with ABA

For the final C, check shorter possible matches

Building the table takes time and extra space

KMP: keep the part that still matches



Keep the text position. Compare B with B.

Keep ABA. Continue to find the match at position 2.

What I expected before testing

Naive could do well when mismatches happen early

KMP should help with long partial matches

KMP setup may matter when a match is immediate

Algorithm	Worst-case time	Auxiliary space
naive	O((n - m + 1)m)	O(1)
KMP	O(n + m)	O(m)

How the comparison worked

Same inputs for both algorithms

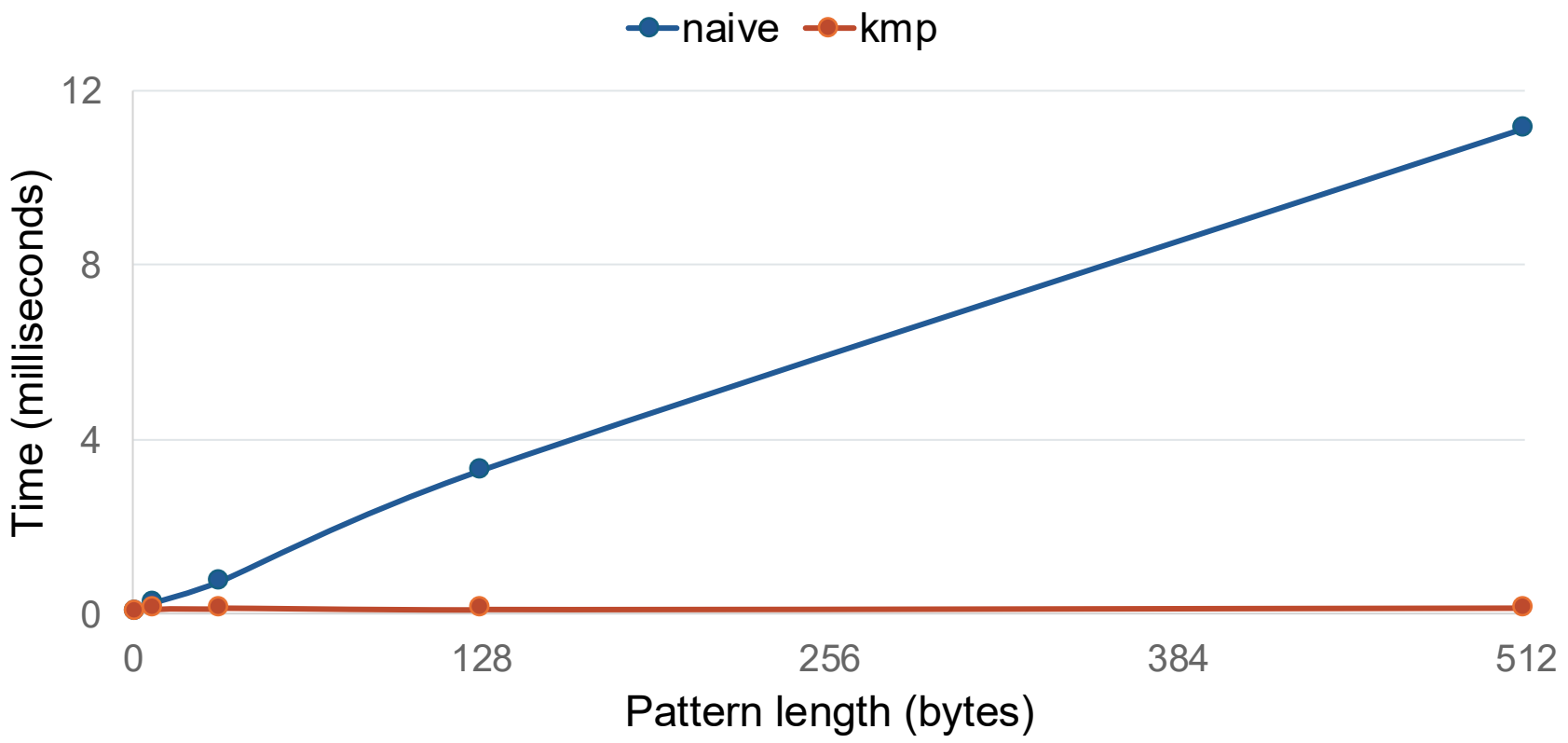
Random, repeated and ordinary text

Repeated timings include KMP table setup

Results come from one Apple M1 Pro Mac

Repeated near-matches: KMP was much faster

65,536-byte repeated text; pattern is absent



104.48x

faster at
m = 512

Medians from three inputs; nine timing batches each

When naïve search keeps up

Match at the beginning (512-byte pattern)

Naive 0.171 µs, KMP 0.506 µs

Ordinary text: both about 209 µs

The input makes a difference

Why did KMP save time?

Naive: about 33.3 million comparisons

KMP: about 132,000, including setup

Fewer comparisons explain much of the difference

Algorithm	Time (microseconds)	Across-instance IQR
naive	11143.000	11135.450-11203.200
KMP	106.656	106.227-107.307

A surprise: similar counts, different times

Both search a long string of a characters

The pattern has an early b, so both reject quickly

Both make about 130,000 comparisons

Algorithm	Equality tests	Time (us)	IQR (us)
naive	130,050	61.691	61.319-61.781
KMP	132,091	107.511	107.373-107.688

KMP took 1.74 times as long in this case

What these results can and cannot tell us

Performance was measured on one computer

Only three input instances for each setting

The prose samples came from one excerpt

Which algorithm is better? It depends

KMP avoids repeated work on difficult inputs

Naive can be faster when a search ends quickly

Next: reuse one pattern across many texts

github.com/egoldman2/substring-search-pa2